Structural Domain Adaptation via Operator-Level Alignment

Author information scrubbed for double-blind reviewing

No Institute Given

Abstract. Unsupervised graph domain adaptation (UGDA) is challenging because distribution shift affects not only node features but also the graph propagation mechanism itself. Existing methods mainly align node embeddings, modify topology, or regularize spectral statistics, but these mechanisms do not directly prevent structure-induced shift from reappearing after each message-passing step. We propose **OPAL**, a filter-centric UGDA framework that aligns *realized graph filters* across domains. OPAL learns a shared feature encoder together with domain-specific graph filters that are reused across layers. The filters are aligned through the response laws they induce on a reference distribution that approximates the feature distribution. Theoretically, we extend Optimal-Transport-based domain adaptation bounds to propagated representations and show that target risk is controlled by a feature-alignment term together with an operator-response term that captures mismatch between the realized propagation operators. Practically, OPAL proposes a filter-response alignment paradigm along with smoothness constraints that arise directly from the theory. The resulting model aligns both semantic representations and structural propagation behavior without graph matching or expensive eigenvalue decomposition, while showing competitive performance with state-of-the-art UGDA baselines.

1 Introduction

Graph-structured data arise in a wide range of domains, including citation networks, molecular structures, recommender systems, and social networks. Graph Neural Networks (GNNs) have become the defacto framework for learning from such relational data by combining node attributes with the underlying graph topology through message passing [23, 42, 24, 25]. This paradigm enables models to exploit dependencies betweein entities and has led to strong performance across many graph-based tasks. However, most GNN models require substantial labeled data to generalize effectively. In many real-world settings, obtaining labels is expensive or impractical. For instance, labeling molecular properties often requires costly simulations or laboratory experiments [4, 7]. These limitations motivate methods that can transfer knowledge across related graph domains.

Domain adaptation addresses this challenge by transferring predictive knowledge from a labeled *source* domain to an unlabeled *target* domain. In graph settings this problem is known as *unsupervised graph domain adaptation* (UGDA),

where the goal is to train a model on a labeled source graph and generalize it to an unlabeled target graph that shares the same label space. The difficulty, compared with standard unsupervised domain adaptation, is that the distribution shift is not only semantic but also structural: changing the graph changes message passing, and changing message passing changes the learned representation [17]. Consequently, models trained on one graph domain often suffer performance degradation when applied to another, where both node attributes and connectivity patterns may differ.

A natural language for structural shift is graph signal processing. A graph neural network layer can be written in terms of a graph shift operator and a graph filter, whose effect is determined by its frequency response evaluated on the spectrum of the underlying graph [33, 10, 17]. This viewpoint has already been productive in UGDA. SpecReg cross-pollinates OT-based domain adaptation bounds with graph filter theory and shows that graph transferability is tied to spectral smoothness and maximum frequency response [38], while DGSDA disentangles attribute shift from topology shift and aligns spectral filters across domains [36]. More recent methods such as A2GNN and TDSS similarly highlight that transfer performance depends strongly on how propagation is performed on the source and target graphs [26, 6]. Recent benchmarking work further shows that UGDA behavior is strongly scenario-dependent and that structural shift becomes increasingly important as the domain gap grows [27].

Unlike feature shift, which enters the model at the input features, structural shift is reintroduced every time message passing is applied. This makes it inherently architecture-dependent. In our view, the relevant object is therefore not only the spectral response but the realized graph filter

$$\mathbf{R}_D = S_{\theta_D}(\widehat{\mathbf{A}}_D),$$

obtained by instantiating a response on a domain-specific graph operator. The spectral response is effectively fixed for standard GNN backbones [25, 41, 35], prescribed by a simple polynomial rule in propagation-based methods [26], and fully learnable in spectral methods [36]. Even if the source and target hidden-feature distributions are well aligned, applying different realized filters can still reintroduce discrepancy. Since the same realized filter is reused at every message-passing layer in our model, any residual structural mismatch is repeatedly applied throughout the network. We therefore aim to align the *operator action* of the realized filters on the feature distributions that actually arise during training, rather than align spectral responses alone.

To address this challenge, we introduce **OPAL** (*Operator-Level Alignment for Graph Domain Adaptation*), a framework that aligns structural propagation behavior across domains by directly comparing their realized graph filters. OPAL learns a shared feature encoder and a shared classifier, together with domain-specific graph filters that remain fixed across layers. Feature alignment is performed on the shared latent feature space, while filter alignment is performed by sampling a reference distribution that is representative of the feature distribution, propagating it through the domain-specific realized filters, and aligning

the resulting response laws. This yields a simple objective that stays close to the OT-based transfer bound while remaining easy to optimize. It also removes the need for graph matching and avoids eigendecomposition in the practical method, because alignment is performed at the level of the learned propagation operators rather than directly on graph structure.

Our contributions are threefold. First, we formulate structural adaptation at the level of realized graph filters and show that the OT-based graph transfer bound naturally decomposes into a feature term and an operator-response term. Second, we derive a practical alignment objective that compares realized filters through the response distributions they induce on a shared feature-supported reference law. Third, we turn these observations into a simple UGDA method that combines source supervision, latent feature alignment, and a Lipschitz-regularized reference-based filter alignment.

2 Related Work

Benchmarking and taxonomy. Recent benchmarking work shows that UGDA performance varies substantially across datasets and transfer scenarios, and that structural shift becomes increasingly important as the domain gap grows [27]. This observation is consistent with the way recent graph adaptation papers organize the field: some methods mainly target node representations, some modify graph structure, and others focus on propagation or spectral behavior. Following this view, we group the most relevant UGDA methods into three families: embedding-alignment methods, structure-aware adaptation methods, and propagation- or spectral-based methods.

Embedding alignment methods. Early UGDA methods mostly follow the standard domain adaptation pipeline of learning domain-invariant node representations. DANE learns transferable node embeddings with a shared GCN encoder and adversarial alignment across networks [41]. AdaGCN extends adversarial adaptation with graph convolution to transfer class information across source and target graphs [9]. UDAGCN combines graph representation learning with domain adaptation by jointly modeling local and global consistency, while GRADE introduces a graph subtree discrepancy to measure graph distribution shift and align cross-network representations [35, 34]. These methods are effective when node-level semantic discrepancy is the dominant challenge, but they largely treat propagation as part of the encoder and do not explicitly isolate the mismatch introduced by domain-specific message passing. In contrast, OPAL keeps a shared feature encoder but adds a separate alignment mechanism at the level of the learned propagation operators.

Structure-aware adaptation methods. A second line of work addresses graph-specific shift more directly by modifying or reweighting structure. StruRW identifies *conditional structure shift* and proposes structural reweighting to correct it through edge-weight recalibration [28]. PairAlign further targets conditional

structure shift together with label shift by combining pairwise edge reweighting and class-balance adjustment [29]. JHGDA studies shifts in hierarchical graph structure and aggregates discrepancy across multiple hierarchy levels, while KBL introduces a knowledge-bridge mechanism to transfer knowledge through an auxiliary bridged graph and a knowledge-enhanced posterior [32, 3]. These methods explicitly exploit graph structure and often improve over embedding-only alignment under larger structural shifts, but they do so by changing graph interactions, adding hierarchical graph modules, or introducing extra transfer graphs rather than aligning the learned propagation operators themselves. Compared with these approaches, OPAL does not alter graph topology or require graph matching; instead, it learns domain-specific filters and aligns their induced behavior directly.

Propagation- and spectral-based methods. A closely related family of methods emphasizes propagation and spectral behavior. A2GNN argues that propagation itself is the key factor in graph adaptation and improves transfer by using asymmetric propagation depth across source and target branches [26]. TDSS smooths the target graph directly to reduce structural inconsistency before adaptation [6]. SpecReg derives graph-specific OT-based transfer bounds and regularizes spectral smoothness and maximum frequency response, while DGSDA disentangles attribute and topology shift and aligns spectral filters across domains [38, 36]. These methods are closest in spirit to our work because they recognize that graph transfer depends on how information is propagated, not only on the final embeddings. Our method differs in the alignment target: instead of modifying the graph, changing propagation depth, or aligning spectral responses, OPAL learns domain-specific realized graph filters and aligns them directly through the response laws they induce on a shared reference distribution supported by the learned features.

Position of our method. Compared with embedding-alignment methods, OPAL explicitly models the structural part of the transfer gap. Compared with structure-editing methods such as StruRW and PairAlign, it does not modify the graph itself. Compared with propagation-depth and spectral methods such as A2GNN, TDSS, SpecReg, and DGSDA, it aligns the *realized* propagation operators on the feature support actually used by the model. This makes OPAL most naturally viewed as an operator-level UGDA method: it uses shared feature learning to reduce semantic shift and reference-based filter alignment to reduce the propagation-induced structural shift identified by our transfer bound.

3 Preliminaries

3.1 Problem definition

We consider UGDA with a labeled source graph $\mathcal{G}^S = (\mathcal{V}^S, \mathcal{E}^S, \mathbf{X}^S, \mathbf{Y}^S)$ and an unlabeled target graph $\mathcal{G}^T = (\mathcal{V}^T, \mathcal{E}^T, \mathbf{X}^T)$. The domains share the label space

$\mathcal{Y}$. As in standard domain adaptation, we adopt the covariate-shift assumption

$$\mathbb{P}_S(Y \mid G) = \mathbb{P}_T(Y \mid G), \qquad \mathbb{P}_S(G) \neq \mathbb{P}_T(G),$$

while acknowledging that label shift and conditional structure shift can occur in practice [31, 28, 29].

Our goal is to learn a predictor that performs well on the target graph by generalizing knowledge from the labeled source graph.

3.2 Graph filter and realized propagation operator

Let $\mathbf{A}_D$ be the adjacency matrix of domain $D \in \{S, T\}$ and define the normalized graph shift operator

$$\widehat{\mathbf{A}}_D = \mathbf{D}_D^{-1/2}(\mathbf{A}_D + \mathbf{I})\mathbf{D}_D^{-1/2}, \tag{1}$$

where $\mathbf{D}_D$ is the degree matrix of $\mathbf{A}_D + \mathbf{I}$. A polynomial graph filter is parameterized by a spectral response

$$h_{\theta_D}(\lambda) = \sum_{k=0}^{K} \theta_{D,k} \lambda^k. \tag{2}$$

Instantiating this response on the graph shift operator yields the corresponding realized graph filter

$$\mathbf{R}_D := S_{\theta_D}(\widehat{\mathbf{A}}_D) = \sum_{k=0}^{K} \theta_{D,k} \widehat{\mathbf{A}}_D^k. \tag{3}$$

We use the term *spectral response* for h_{θ_D} and *realized graph filter* for $\mathbf{R}_D$. The former is graph structure-independent; the latter is the domain-specific linear operator that actually propagates information on the graph.

3.3 Shared encoder and domain-specific propagation

OPAL learns a shared feature encoder f_ϕ together with domain-specific realized filters $\mathbf{R}_S$ and $\mathbf{R}_T$. Let $\mathbf{H}_D^{(\ell)}$ denote the hidden representation at layer ℓ. We first form

$$\mathbf{H}_D^{(0)} = f_\phi(\mathbf{X}_D), \tag{4}$$

and then propagate through L layers according to

$$\mathbf{H}_D^{(\ell+1)} = \sigma\Big(\mathbf{R}_D \mathbf{H}_D^{(\ell)} \mathbf{W}^{(\ell)}\Big), \qquad \ell = 0, \ldots, L-1. \tag{5}$$

The transformation matrices $\mathbf{W}^{(\ell)}$ are shared across domains, while the realized filter $\mathbf{R}_D$ is domain-specific and reused in every message-passing layer within domain D. The final prediction is obtained through a shared classifier

$$\hat{\mathbf{Y}}_D = g_\psi(\mathbf{H}_D^{(L)}). \tag{6}$$

No additional domain-specific propagation is used at the classification stage.

4 Operator-Level Shift

Why align realized graph filters? Recent UGDA work already suggests that structure should not be treated as an ordinary nuisance variable. SpecReg links transfer performance to spectral properties of graph encoders [38], while DGSDA explicitly argues for disentangling attribute and topology shift and aligns spectral filters across domains [36]. A2GNN changes the effective propagation depth across domains, and TDSS smooths the target graph directly to mitigate structural inconsistency [26, 6]. StruRW and PairAlign reweight graph structure to reduce conditional mismatch [28, 29]. These approaches all indicate that the core issue in UGDA is not only what features are learned, but how those features are propagated.

Our perspective is that the relevant object is the realized graph filter $\mathbf{R}_D = S_{\theta_D}(\widehat{\mathbf{A}}_D)$, not the spectral response in isolation. Even when source and target hidden-feature laws are aligned, a propagation step can reintroduce discrepancy if $\mathbf{R}_S$ and $\mathbf{R}_T$ act differently. Since OPAL reuses the same realized filter at every message-passing layer within a domain, any remaining operator mismatch is repeatedly applied as depth increases.

A decomposition of structural mismatch. For theory, we follow existing graph adaptation analyses and pad source and target graphs with isolated nodes to a common size N [43]. Let $\widetilde{\mathbf{A}}_T = P^\star \widehat{\mathbf{A}}_T P^{\star\top}$ denote the optimally permuted target shift operator under the matching metric used in the analysis, and define

$$\mathbf{R}_S = S_{\theta_S}(\widehat{\mathbf{A}}_S), \qquad \widetilde{\mathbf{R}}_T = S_{\theta_T}(\widetilde{\mathbf{A}}_T).$$

The realized-filter mismatch can then be decomposed as

$$\|\mathbf{R}_S - \widetilde{\mathbf{R}}_T\|_{\mathrm{op}} \le \underbrace{\|S_{\theta_S}(\widehat{\mathbf{A}}_S) - S_{\theta_S}(\widetilde{\mathbf{A}}_T)\|_{\mathrm{op}}}_{\text{graph perturbation term}} + \underbrace{\|S_{\theta_S}(\widetilde{\mathbf{A}}_T) - S_{\theta_T}(\widetilde{\mathbf{A}}_T)\|_{\mathrm{op}}}_{\text{response mismatch term}}. \tag{7}$$

The first term measures how much the same filter changes when instantiated on two different graphs; the second measures how much the two learned filters differ on the same graph.

The graph perturbation term is governed by graph-filter stability. In particular, if h_{θ_S} is integral Lipschitz with constant C_{θ_S} and $\widehat{\mathbf{A}}_S$ and $\widetilde{\mathbf{A}}_T$ differ by a relative perturbation of size ε, then

$$\|S_{\theta_S}(\widehat{\mathbf{A}}_S) - S_{\theta_S}(\widetilde{\mathbf{A}}_T)\|_{\mathrm{op}} \le 2C_{\theta_S}(1 + \delta\sqrt{N})\,\varepsilon + O(\varepsilon^2), \tag{8}$$

where δ is the eigenvector-misalignment term induced by the perturbation [17]. This shows that regularizing the filter coefficients can control the perturbation-sensitive part of the mismatch, but the response mismatch term must still be learned away.

From feature shift to propagation shift. Let P_S and P_T denote the source and target hidden-feature laws at the encoder output, i.e., the laws of rows of $\mathbf{H}_S^{(0)}$ and $\mathbf{H}_T^{(0)}$. Define the propagated laws

$$P_S^{\mathbf{R}} = (\mathbf{R}_S)_{\#}P_S, \qquad P_T^{\mathbf{R}} = (\widetilde{\mathbf{R}}_T)_{\#}P_T.$$

We measure operator mismatch on the feature support through

$$\mathcal{D}_{\mathrm{op}}(\mathbf{R}_S, \widetilde{\mathbf{R}}_T; P_S) := \mathbb{E}_{U\sim P_S}\|(\mathbf{R}_S - \widetilde{\mathbf{R}}_T)U\|_2. \tag{9}$$

Proposition 1 (Propagated Wasserstein discrepancy). *With the above notation,*

$$W_1(P_S^{\mathbf{R}}, P_T^{\mathbf{R}}) \le \mathcal{D}_{\mathrm{op}}(\mathbf{R}_S, \widetilde{\mathbf{R}}_T; P_S) + \|\widetilde{\mathbf{R}}_T\|_{\mathrm{op}}\, W_1(P_S, P_T). \tag{10}$$

Eq. (10) is the key structural statement. It shows that propagation mismatch after filtering is controlled by a standard feature-divergence term and a new operator-response term. Therefore, feature alignment alone is not sufficient. If the realized filters remain different, message passing can recreate discrepancy even after the hidden features are aligned.

Operator-level transfer bound. We now plug Eq. (10) into the OT-based domain adaptation bound used in prior graph DA work [31, 38]. Let g be a C_g-Lipschitz classifier acting on propagated representations, and let $\mathcal{H}$ be a bounded hypothesis class of pseudo-dimension d. Then the target risk can be bounded as follows.

Theorem 1 (Operator-response adaptation bound). *Assume the covariate-shift condition holds on propagated representations. Then, with probability at least $1-\delta$ over the labeled source sample,*

$$\begin{aligned}\epsilon_T(g) \le \widehat{\epsilon}_S(g) &+ \sqrt{\frac{4d}{n_S}\log\frac{en_S}{d} + \frac{1}{n_S}\log\frac{1}{\delta}} \\ &+ 2C_g\,\mathcal{D}_{\mathrm{op}}(\mathbf{R}_S, \widetilde{\mathbf{R}}_T; P_S) + 2C_g\,\|\widetilde{\mathbf{R}}_T\|_{\mathrm{op}} W_1(P_S, P_T) + \omega,\end{aligned} \tag{11}$$

where ω is the usual joint best-hypothesis term.

Theorem 1 makes explicit why operator-level alignment matters. The target risk is controlled by *both* feature mismatch and operator mismatch. From this perspective, methods such as StruRW and PairAlign reduce structural shift by changing edge weights, while A2GNN changes how much propagation is applied on each domain, and TDSS smooths the target structure directly. OPAL differs in that it operates directly on the learned realized filters. Rather than editing the graph or manually adjusting propagation depth, it learns domain-specific propagation operators and aligns them on the feature support that the model actually uses.

5 Proposed Method

Overview. OPAL combines three objectives: a source classification loss, a feature alignment loss, and a filter alignment loss. The first preserves task semantics on the source graph. The second reduces discrepancy between source and target hidden features. The third aligns the realized graph filters through the response laws they induce on bootstrap probes sampled from a common empirical feature distribution. A light integral-Lipschitz regularizer is included inside the filter alignment term to stabilize the learned propagation operators.

5.1 Source classification and feature alignment

The source classification loss is

$$\mathcal{L}_{\mathrm{cls}} = \frac{1}{|\mathcal{V}_L^S|} \sum_{v \in \mathcal{V}_L^S} \mathrm{CE}\left(\hat{\mathbf{Y}}_S(v), \mathbf{Y}^S(v)\right). \tag{12}$$

To reduce semantic discrepancy, we align the hidden-feature distributions at the encoder output. Define the empirical feature law

$$\widehat{P}_D = \frac{1}{n_D} \sum_{i=1}^{n_D} \delta_{\mathbf{H}_D^{(0)}(i,:)}. \tag{13}$$

The feature alignment loss is then

$$\mathcal{L}_{\mathrm{feat}} = \mathrm{Sink}_\varepsilon(\widehat{P}_S, \widehat{P}_T). \tag{14}$$

This term corresponds directly to the feature-divergence part of the transfer bound in Eq. (11).

5.2 Bootstrap-based filter alignment

The remaining part of the transfer bound is the operator-response discrepancy. In practice, we estimate it on a shared empirical feature support.

We first form a pooled reference feature law by combining the source and target encoder outputs:

$$\widehat{P}_{\mathrm{ref}} = \frac{1}{n_S + n_T} \left(\sum_{i=1}^{n_S} \delta_{\mathbf{H}_S^{(0)}(i,:)} + \sum_{j=1}^{n_T} \delta_{\mathbf{H}_T^{(0)}(j,:)} \right). \tag{15}$$

For each domain $D \in \{S, T\}$, we then construct a bootstrap probe matrix $\mathbf{B}_D \in \mathbb{R}^{n_D \times d_h}$ by sampling each row independently from $\widehat{P}_{\mathrm{ref}}$, where d_h is the hidden dimension:

$$\mathbf{B}_D(i,:) \overset{\text{i.i.d.}}{\sim} \widehat{P}_{\mathrm{ref}}, \qquad i = 1, \ldots, n_D. \tag{16}$$

By construction, the source and target probes are sampled from the same feature distribution. Therefore, any difference between the propagated probe responses comes from the realized filters rather than from differences in the input law.

The probe responses are

$$\mathbf{Q}_D = \mathbf{R}_D \mathbf{B}_D. \tag{17}$$

We define the corresponding empirical response law as

$$\widehat{\rho}_D = \frac{1}{n_D} \sum_{i=1}^{n_D} \delta_{\mathbf{Q}_D(i,:)}. \tag{18}$$

The bootstrap-based response discrepancy is then measured with Sinkhorn divergence:

$$\mathcal{L}_{\text{resp}} = \text{Sink}_\varepsilon(\widehat{\rho}_S, \widehat{\rho}_T). \tag{19}$$

This is the practical counterpart of the operator-response term in Eq. (11). Instead of comparing the full operators directly, we compare their actions on a common bootstrap feature support. Since the loss only depends on empirical row measures, it is permutation invariant and does not require graph matching.

5.3 Integral-Lipschitz regularization

To stabilize the learned realized filters, we add a lightweight regularizer on the polynomial coefficients. Because the spectrum of $\widehat{\mathbf{A}}_D$ lies in $[-1, 1]$, the integral-Lipschitz seminorm of a polynomial response can be controlled directly from its coefficients.

Proposition 2 (Coefficient control of integral-Lipschitz seminorm). *If* $q(\lambda) = \sum_{k=0}^{K_q} a_k \lambda^k$ *and* $|\lambda| \le 1$, *then*

$$\sup_{|\lambda| \le 1} |\lambda q'(\lambda)| \le \sum_{k=1}^{K_q} k|a_k|. \tag{20}$$

Motivated by Proposition 2, we define

$$\mathcal{L}_{\text{Lip}} = \sum_{D \in \{S,T\}} \left(\sum_{k=1}^{K} k|\theta_{D,k}| \right)^2. \tag{21}$$

Its role is not to impose explicit spectral smoothing, but to keep the learned propagation operators stable under structural perturbations.

We group response alignment and coefficient regularization into a single filter alignment term:

$$\mathcal{L}_{\text{filt}} = \mathcal{L}_{\text{resp}} + \lambda_{\text{Lip}} \mathcal{L}_{\text{Lip}}. \tag{22}$$

5.4 Overall objective

The final training objective is

$$\min_{\phi,\psi,\theta_S,\theta_T,\{\mathbf{W}^{(\ell)}\}_{\ell=0}^{L-1}} \quad \mathcal{L}_{\text{cls}} + \lambda_{\text{feat}}\mathcal{L}_{\text{feat}} + \lambda_{\text{filt}}\mathcal{L}_{\text{filt}}. \tag{23}$$

Thus OPAL has exactly three functional pieces: (i) source supervision, (ii) latent feature alignment, and (iii) filter alignment on a shared bootstrap feature support. The first two reduce semantic shift; the third directly targets propagation mismatch.

Complexity. For polynomial filters, multiplying a d-dimensional feature matrix by $\mathbf{R}_D$ costs $O(K|\mathcal{E}_D|d)$ using sparse matrix multiplications. The bootstrap alignment adds one extra filtered pass through $\mathbf{R}_S$ and $\mathbf{R}_T$ per iteration. Since the realized filters are learned once per domain and reused across layers, OPAL is computationally simpler than methods that modify graph structure directly or learn separate propagation operators at every layer.

6 Experiments

6.1 Main performance comparison

We evaluate OPAL on standard UGDA node-classification benchmarks over citation, social, and airport graphs. We report our results mean ± standard deviation over five seeds and paired significance tests against the strongest recent baselines [27].

Datasets. We use ArnetMiner, BlogCatalog, Twitch, and Airport. From Twitch we follow prior work and use the three largest domains: England (EN), Germany (DE), and France (FR).

Baselines. The baselines cover embedding-alignment, structure-aware, and propagation/spectral UGDA methods. Concretely, we compare against A2GNN [26], AdaGCN [9], DANE [41], DGSDA [36], GRADE [34], JHGDA [32], KBL [3], PairAlign [29], SpecReg [38], StruRW [28], TDSS [6], and UDAGCN [35]. This set covers the main families discussed in Section 2 and provides a stronger comparison than limiting evaluation to embedding-alignment baselines only.

Implementation details. To address reproducibility concerns, the revised variant should report: number of propagation layers L, polynomial order $K \in \{2, 3, 4, 5\}$, hidden dimension d_h, Sinkhorn regularization $\varepsilon \in \{0.01, 0.05, 0.1, 0.5\}$, and trade-off weights λ_{feat}, λ_{filt}, and λ_{Lip}. Bootstrap probes are resampled at every training iteration. For all datasets, we run experiments for 5 seeds.

Results. Table **??** shows kasu

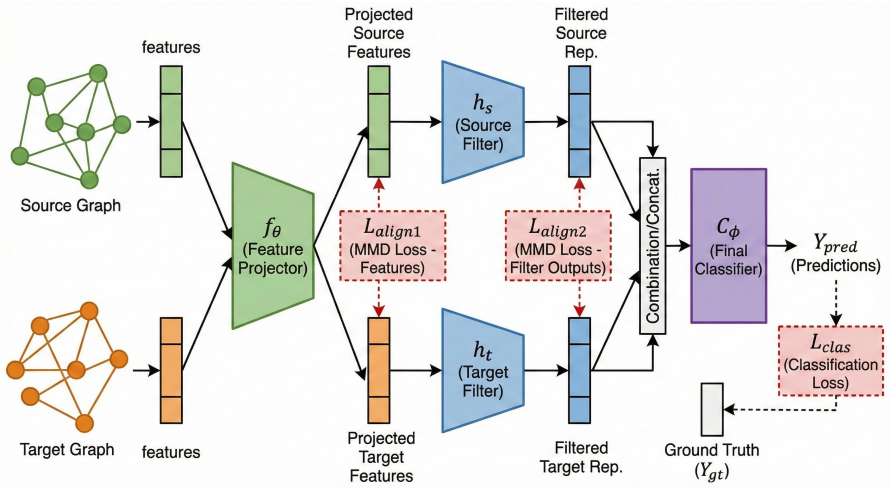

Fig. 1. Overview of OPAL. A shared encoder produces hidden node representations, while each domain uses its own realized graph filter $\mathbf{R}_D = S_{\theta_D}(\widehat{\mathbf{A}}_D)$, reused across layers, to propagate information. Training combines source supervision, Sinkhorn alignment of latent feature distributions, and bootstrap-based alignment of filter responses computed on a common empirical feature support. This directly targets both semantic shift and propagation-induced structural shift without modifying the graph topology.

6.2 Bootstrap probe ablation

6.3 Robustness to structural perturbation

6.4 Understanding realized filters as induced propagation graphs

6.5 Ablation and hyperparameter study

7 Conclusion

This paper argues that structural adaptation in UGDA should be formulated at the level of *realized graph filters*. Once the notation is organized around graph shift operators, spectral responses, and realized propagation operators, the key issue becomes clear: even after hidden features are aligned, domain-specific message passing can recreate discrepancy unless the realized filters themselves are controlled. OPAL implements this perspective with a shared encoder, domain-specific filters reused across layers, Sinkhorn feature alignment, and reference-based filter alignment on a common empirical feature support. A lightweight

Table 1. Micro-F1 (%) on Airport, Blog, Citation, and Twitch datasets. Each entry reports mean and standard deviation across seeds. Red, Orange, and Blue indicate the top 3 results; **bold** indicates the 4th-best result. Avg. Rank is computed over all available transfer tasks, with lower being better.

Models	Airport						Blog		Avg. Rank
	B→E	B→U	E→B	E→U	U→B	U→E	B1→B2	B2→B1	
A2GNN	50.54 ± 0.38	55.24 ± 0.55	57.51 ± 2.89	55.38 ± 0.15	62.60 ± 3.82	53.22 ± 0.63	21.12 ± 3.76	20.01 ± 3.77	5.50
AdaGCN	55.22 ± 0.63	**51.54 ± 1.81**	**68.96 ± 1.92**	54.48 ± 1.85	63.87 ± 0.88	52.46 ± 1.51	**46.22 ± 3.54**	50.03 ± 3.89	5.28
DANE	32.58 ± 0.43	43.28 ± 1.70	45.04 ± 0.76	50.28 ± 3.45	40.20 ± 1.59	34.92 ± 1.18	17.35 ± 0.02	17.00 ± 1.05	11.59
DGSDA	50.71 ± 2.19	56.95 ± 0.87	**68.96 ± 1.17**	51.26 ± 0.15	66.92 ± 1.17	51.55 ± 0.77	61.49 ± 1.71	57.14 ± 5.70	2.78
GRADE	55.47 ± 0.14	49.75 ± 0.42	77.86 ± 0.76	50.28 ± 0.17	60.81 ± 2.89	**49.21 ± 1.53**	48.60 ± 3.37	**49.49 ± 1.84**	4.97
JHGDA	52.38 ± 0.50	48.57 ± 0.25	68.70 ± 1.32	46.05 ± 0.67	53.18 ± 1.92	46.37 ± 1.40	17.30 ± 0.50	20.88 ± 6.12	7.91
KBL	**53.47 ± 0.95**	45.27 ± 1.24	70.48 ± 1.17	46.02 ± 0.13	55.98 ± 0.44	41.60 ± 0.90	38.36 ± 6.66	41.22 ± 3.74	6.56
PairAlign	40.69 ± 3.42	45.91 ± 5.88	50.89 ± 0.88	40.06 ± 0.80	59.54 ± 2.75	42.44 ± 0.38	43.06 ± 1.11	45.19 ± 0.59	8.75
SpecReg	54.30 ± 1.53	42.52 ± 3.93	65.90 ± 3.09	46.58 ± 5.05	53.18 ± 18.33	41.10 ± 3.04	20.38 ± 1.64	20.36 ± 0.35	7.22
StruRW	34.00 ± 1.26	39.66 ± 0.87	58.02 ± 1.53	42.38 ± 2.03	**63.10 ± 4.47**	39.43 ± 0.63	41.38 ± 0.37	42.84 ± 0.60	10.56
TDSS	49.79 ± 1.24	46.41 ± 1.26	72.26 ± 3.09	48.54 ± 6.77	45.80 ± 1.32	44.44 ± 0.38	18.63 ± 2.25	18.64 ± 4.39	9.44
UDAGCN	48.96 ± 1.04	47.45 ± 1.39	60.81 ± 4.85	44.82 ± 0.59	53.94 ± 5.73	41.02 ± 2.63	26.38 ± 5.84	29.52 ± 2.85	7.50
OPAL (ours)	50.96 ± 1.26	57.23 ± 0.25	62.09 ± 0.44	**50.90 ± 1.05**	70.48 ± 2.20	47.54 ± 0.38	67.79 ± 0.55	65.33 ± 2.03	2.94

Models	Citation						Twitch		Avg. Rank
	A→C	A→D	C→A	C→D	D→A	D→C	DE→EN	EN→DE	
A2GNN	81.75 ± 0.06	**75.98 ± 0.88**	75.37 ± 0.20	76.81 ± 0.79	72.95 ± 0.62	80.74 ± 1.19	56.46 ± 0.30	61.29 ± 0.32	5.50
AdaGCN	75.34 ± 0.63	70.84 ± 1.49	69.37 ± 0.53	74.29 ± 0.41	62.13 ± 0.38	71.13 ± 1.25	59.53 ± 0.11	63.95 ± 0.08	5.28
DANE	67.36 ± 4.23	62.18 ± 0.80	60.95 ± 1.77	64.82 ± 1.54	56.20 ± 1.40	65.70 ± 2.39	58.65 ± 0.33	63.35 ± 1.02	11.59
DGSDA	82.57 ± 0.42	76.29 ± 0.36	75.83 ± 0.09	78.85 ± 0.53	72.85 ± 0.13	81.67 ± 0.34	60.14 ± 0.09	63.93 ± 0.04	2.78
GRADE	76.63 ± 0.75	68.62 ± 0.84	70.70 ± 0.05	73.98 ± 0.03	65.59 ± 1.69	73.24 ± 1.72	59.73 ± 0.04	65.22 ± 0.01	4.97
JHGDA	75.03 ± 0.38	70.59 ± 0.08	67.88 ± 0.45	72.42 ± 0.23	63.97 ± 0.76	72.85 ± 0.11	58.86 ± 0.26	64.45 ± 0.28	7.91
KBL	78.24 ± 0.55	69.37 ± 0.10	70.74 ± 0.22	75.53 ± 0.56	63.61 ± 0.38	74.42 ± 0.96	**59.74 ± 0.06**	65.03 ± 0.91	6.56
PairAlign	69.21 ± 0.39	64.67 ± 1.36	63.51 ± 1.61	69.06 ± 0.49	58.15 ± 0.35	66.99 ± 1.07	60.00 ± 0.27	**64.14 ± 0.32**	8.75
SpecReg	**81.25 ± 1.46**	77.49 ± 0.61	**73.81 ± 1.25**	76.28 ± 1.57	**71.59 ± 1.01**	**78.92 ± 1.27**	55.67 ± 1.16	61.43 ± 0.05	7.22
StruRW	65.99 ± 1.96	61.74 ± 1.83	63.02 ± 0.92	65.94 ± 0.77	57.99 ± 1.01	66.17 ± 1.84	57.37 ± 2.44	61.33 ± 0.63	10.56
TDSS	75.47 ± 1.63	67.13 ± 3.92	63.49 ± 3.43	**76.80 ± 1.79**	56.07 ± 4.51	66.65 ± 6.35	54.56 ± 0.00	60.45 ± 0.00	9.44
UDAGCN	80.07 ± 1.40	73.20 ± 0.69	71.99 ± 0.58	76.78 ± 1.24	65.02 ± 4.95	76.29 ± 2.42	59.29 ± 0.25	62.43 ± 0.67	7.50
OPAL (ours)	82.19 ± 0.42	76.56 ± 0.14	75.09 ± 0.53	78.98 ± 0.31	72.39 ± 0.36	81.84 ± 0.04	60.49 ± 0.21	63.90 ± 0.16	2.94

integral-Lipschitz regularizer further stabilizes the learned propagation operators without imposing explicit spectral smoothing. Empirically, the resulting method remains competitive with recent structure-aware UGDA baselines while avoiding graph matching and direct topology modification.

A natural next step is source-free adaptation. Since the filter alignment loss compares realized propagation operators through their responses on feature-supported probes rather than through raw source features, the same principle may remain applicable even when source data are unavailable at adaptation time.

References

1. Ben-David, S., Blitzer, J., Crammer, K., Pereira, F.: Analysis of representations for domain adaptation. Advances in neural information processing systems **19** (2006)
2. Bhattacharya, R., Manwani, N.: Edgemask-dg*: Learning domain-invariant graph structures via adversarial edge masking. arXiv preprint arXiv:2602.05571 (2026)
3. Bi, W., Cheng, X., Xu, B., Sun, X., Xu, L., Shen, H.: Bridged-gnn: Knowledge bridge learning for effective knowledge transfer. In: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management. pp. 99–109 (2023)
4. Butler, K.T., Davies, D.W., Cartwright, H., Isayev, O., Walsh, A.: Machine learning for molecular and materials science. Nature **559**(7715), 547–555 (2018)

5. Chen, W., Guo, X., Li, S., Zhang, Z., Zhong, Y., Zhuang, F., et al.: Learning adaptive distribution alignment with neural characteristic function for graph domain adaptation. arXiv preprint arXiv:2602.10489 (2026)
6. Chen, W., Ye, G., Wang, Y., Zhang, Z., Zhang, L., Wang, D., Zhang, Z., Zhuang, F.: Smoothness really matters: A simple yet effective approach for unsupervised graph domain adaptation. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 39, pp. 15875–15883 (2025)
7. Choo, H.Y., Wee, J., Shen, C., Xia, K.: Fingerprint-enhanced graph attention network (fingat) model for antibiotic discovery. Journal of Chemical Information and Modeling **63**(10), 2928–2935 (2023)
8. Cuturi, M.: Sinkhorn distances: Lightspeed computation of optimal transport. Advances in neural information processing systems **26** (2013)
9. Dai, Q., Wu, X.M., Xiao, J., Shen, X., Wang, D.: Graph transfer learning via adversarial domain adaptation with graph convolution. IEEE Transactions on Knowledge and Data Engineering **35**(5), 4908–4922 (2022)
10. Defferrard, M., Bresson, X., Vandergheynst, P.: Convolutional neural networks on graphs with fast localized spectral filtering. Advances in neural information processing systems **29** (2016)
11. Deshpande, Y., Sen, S., Montanari, A., Mossel, E.: Contextual stochastic block models. Advances in neural information processing systems **31** (2018)
12. Donnat, C., Zitnik, M., Hallac, D., Leskovec, J.: Learning structural node embeddings via diffusion wavelets. In: Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining. pp. 1320–1329 (2018)
13. Fang, R., Li, B., Kang, Z., Zeng, Q., Dashtbayaz, N.H., Pu, R., Wang, B., Ling, C.: On the benefits of attribute-driven graph domain adaptation. arXiv preprint arXiv:2502.06808 (2025)
14. Fang, R., Li, B., Zhao, J., Pu, R., Zeng, Q., Xu, G., Ling, C., Wang, B.: Homophily enhanced graph domain adaptation. arXiv preprint arXiv:2505.20089 (2025)
15. Fang, R., Wang, S., Pu, R., Zeng, Q., Zheng, H., Wang, Z., Cai, J., Mei, Z., Tang, S., Ling, C., et al.: Graph domain adaptation via homophily-agnostic reconstructing structure. arXiv preprint arXiv:2602.07573 (2026)
16. Feydy, J., Séjourné, T., Vialard, F.X., Amari, S.i., Trouvé, A., Peyré, G.: Interpolating between optimal transport and mmd using sinkhorn divergences. In: The 22nd international conference on artificial intelligence and statistics. pp. 2681–2690. PMLR (2019)
17. Gama, F., Bruna, J., Ribeiro, A.: Stability properties of graph neural networks. IEEE Transactions on Signal Processing **68**, 5680–5695 (2020)
18. Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., March, M., Lempitsky, V.: Domain-adversarial training of neural networks. Journal of machine learning research **17**(59), 1–35 (2016)
19. Genevay, A., Chizat, L., Bach, F., Cuturi, M., Peyré, G.: Sample complexity of sinkhorn divergences. In: The 22nd international conference on artificial intelligence and statistics. pp. 1574–1583. PMLR (2019)
20. Gretton, A., Borgwardt, K.M., Rasch, M.J., Schölkopf, B., Smola, A.: A kernel two-sample test. The journal of machine learning research **13**(1), 723–773 (2012)
21. Hou, Z., Li, H., Cen, Y., Tang, J., Dong, Y.: Graphalign: Pretraining one graph neural network on multiple graphs via feature alignment. arXiv preprint arXiv:2406.02953 (2024)
22. Huang, R., Xu, J., Jiang, X., An, R., Yang, Y.: Can modifying data address graph domain adaptation? In: Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining. pp. 1131–1142 (2024)

23. Jiang, X., Zhuang, D., Zhang, X., Chen, H., Luo, J., Gao, X.: Uncertainty quantification via spatial-temporal tweedie model for zero-inflated and long-tail travel demand prediction. In: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management. pp. 3983–3987 (2023)
24. Ju, W., Fang, Z., Gu, Y., Liu, Z., Long, Q., Qiao, Z., Qin, Y., Shen, J., Sun, F., Xiao, Z., et al.: A comprehensive survey on deep graph representation learning. Neural Networks **173**, 106207 (2024)
25. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907 (2016)
26. Liu, M., Fang, Z., Zhang, Z., Gu, M., Zhou, S., Wang, X., Bu, J.: Rethinking propagation for unsupervised graph domain adaptation. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 38, pp. 13963–13971 (2024)
27. Liu, M., Zhang, Z., Tang, J., Bu, J., He, B., Zhou, S.: Revisiting, benchmarking and understanding unsupervised graph domain adaptation. Advances in Neural Information Processing Systems **37**, 89408–89436 (2024)
28. Liu, S., Li, T., Feng, Y., Tran, N., Zhao, H., Qiu, Q., Li, P.: Structural re-weighting improves graph domain adaptation. In: International conference on machine learning. pp. 21778–21793. PMLR (2023)
29. Liu, S., Zou, D., Zhao, H., Li, P.: Pairwise alignment improves graph domain adaptation. arXiv preprint arXiv:2403.01092 (2024)
30. Pilancı, M., Vural, E.: Domain adaptation on graphs by learning aligned graph bases. IEEE Transactions on Knowledge and Data Engineering **34**(2), 587–600 (2020)
31. Redko, I., Habrard, A., Sebban, M.: Theoretical analysis of domain adaptation with optimal transport. In: Joint European Conference on Machine Learning and Knowledge Discovery in Databases. pp. 737–753. Springer (2017)
32. Shi, B., Wang, Y., Guo, F., Shao, J., Shen, H., Cheng, X.: Improving graph domain adaptation with network hierarchy. In: Proceedings of the 32nd ACM international conference on information and knowledge management. pp. 2249–2258 (2023)
33. Shuman, D.I., Narang, S.K., Frossard, P., Ortega, A., Vandergheynst, P.: The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. IEEE signal processing magazine **30**(3), 83–98 (2013)
34. Wu, J., He, J., Ainsworth, E.: Non-iid transfer learning on graphs. In: Proceedings of the AAAI conference on artificial intelligence. vol. 37, pp. 10342–10350 (2023)
35. Wu, M., Pan, S., Zhou, C., Chang, X., Zhu, X.: Unsupervised domain adaptive graph convolutional networks. In: Proceedings of the web conference 2020. pp. 1457–1467 (2020)
36. Yang, L., Chen, X., Zhuo, J., Jin, D., Wang, C., Cao, X., Wang, Z., Guo, Y.: Disentangled graph spectral domain adaptation. In: Forty-second International Conference on Machine Learning (2025)
37. Yang, Z.R., Han, J., Wang, C.D., Liu, H.: Graphlora: Structure-aware contrastive low-rank adaptation for cross-graph transfer learning. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1. pp. 1785–1796 (2025)
38. You, Y., Chen, T., Wang, Z., Shen, Y.: Graph domain adaptation via theory-grounded spectral regularization. In: The eleventh international conference on learning representations (2023)
39. Zellinger, W., Grubinger, T., Lughofer, E., Natschläger, T., Saminger-Platz, S.: Central moment discrepancy (cmd) for domain-invariant representation learning. arXiv preprint arXiv:1702.08811 (2017)

40. Zeng, Z., Qiu, R., Bao, W., Wei, T., Lin, X., Yan, Y., Abdelzaher, T.F., Han, J., Tong, H.: Pave your own path: Graph gradual domain adaptation on fused gromov-wasserstein geodesics. arXiv preprint arXiv:2505.12709 (2025)
41. Zhang, Y., Song, G., Du, L., Yang, S., Jin, Y.: Dane: Domain adaptive network embedding. arXiv preprint arXiv:1906.00684 (2019)
42. Zhou, Y., Zheng, H., Huang, X., Hao, S., Li, D., Zhao, J.: Graph neural networks: Taxonomy, advances, and trends. ACM Transactions on Intelligent Systems and Technology (TIST) **13**(1), 1–54 (2022)
43. Zhu, Q., Yang, C., Xu, Y., Wang, H., Zhang, C., Han, J.: Transfer learning of graph neural networks with ego-graph information maximization. Advances in Neural Information Processing Systems **34**, 1766–1779 (2021)
44. Zhuang, Y., Shi, C., Yang, C., Zhuang, F., Song, Y.: Semantic-specific hierarchical alignment network for heterogeneous graph adaptation. In: Joint European Conference on Machine Learning and Knowledge Discovery in Databases. pp. 335–350. Springer (2021)

A Appendix: Additional + Proofs

A.1 Additional Experiment 1: Distribution shift and performance degradation

In this experiment, we analyze the relationship between the distribution shift due to domains, and the performance degradation of model trained on source domain and tested on target domain. We use Wasserstein distance to measure the distribution shift, and accuracy drop to measure the performance degradation. We find that there is a positive correlation between the two, which suggests that larger distribution shift leads to more performance degradation. This motivates the need for domain adaptation methods that can reduce the distribution shift and improve the performance on target domain.

A.2 Proof of Proposition 1

Proof. Let π be any coupling between P_S and P_T. The pushforward of π by $(u, v) \mapsto (\mathbf{R}_S u, \widetilde{\mathbf{R}}_T v)$ is a coupling between $P_S^{\mathbf{R}}$ and $P_T^{\mathbf{R}}$. Therefore,

$$W_1(P_S^{\mathbf{R}}, P_T^{\mathbf{R}}) \leq \int \|\mathbf{R}_S u - \widetilde{\mathbf{R}}_T v\|_2 \, d\pi(u, v).$$

Add and subtract $\widetilde{\mathbf{R}}_T u$:

$$\|\mathbf{R}_S u - \widetilde{\mathbf{R}}_T v\|_2 \leq \|(\mathbf{R}_S - \widetilde{\mathbf{R}}_T)u\|_2 + \|\widetilde{\mathbf{R}}_T(u - v)\|_2.$$

Using $\|\widetilde{\mathbf{R}}_T(u - v)\|_2 \leq \|\widetilde{\mathbf{R}}_T\|_{\text{op}} \|u - v\|_2$ and integrating with respect to π yields

$$W_1(P_S^{\mathbf{R}}, P_T^{\mathbf{R}}) \leq \mathbb{E}_{u \sim P_S} \|(\mathbf{R}_S - \widetilde{\mathbf{R}}_T)u\|_2 + \|\widetilde{\mathbf{R}}_T\|_{\text{op}} \int \|u - v\|_2 \, d\pi(u, v).$$

Finally, minimize over all couplings π to obtain Eq. (10).

A.3 Proof of Theorem 1

Proof. Apply the Redko-style OT DA bound at the propagated representation level, i.e., to the pair of distributions $(P_S^{\mathbf{R}}, P_T^{\mathbf{R}})$. This yields

$$\epsilon_T(g) \leq \widehat{\epsilon}_S(g) + \sqrt{\frac{4d}{n_S}\log\frac{en_S}{d} + \frac{1}{n_S}\log\frac{1}{\delta}} + 2C_g W_1(P_S^{\mathbf{R}}, P_T^{\mathbf{R}}) + \omega.$$

Now invoke Proposition 1:

$$W_1(P_S^{\mathbf{R}}, P_T^{\mathbf{R}}) \leq \mathcal{D}_{\mathrm{op}}(\mathbf{R}_S, \widetilde{\mathbf{R}}_T; P_S) + \|\widetilde{\mathbf{R}}_T\|_{\mathrm{op}} W_1(P_S, P_T).$$

Substituting this inequality into the OT bound proves Eq. (11).

A.4 Proof of Proposition 2

Proof. Let $q(\lambda) = \sum_{k=0}^{K_q} a_k \lambda^k$. Then

$$q'(\lambda) = \sum_{k=1}^{K_q} k a_k \lambda^{k-1}.$$

Multiplying by λ gives

$$\lambda q'(\lambda) = \sum_{k=1}^{K_q} k a_k \lambda^k.$$

Hence, for $|\lambda| \leq 1$,

$$|\lambda q'(\lambda)| \leq \sum_{k=1}^{K_q} k|a_k||\lambda|^k \leq \sum_{k=1}^{K_q} k|a_k|.$$

Taking the supremum over $|\lambda| \leq 1$ proves Eq. (20).